



Octane Security Report

Security Analysis of Superform: v2-periphery (main)
11/27/2025

About Octane

Octane is a security intelligence platform purpose-built for modern software teams. We help engineering and security teams identify, investigate, and triage vulnerabilities faster—without disrupting the development process.

Combining automated static analysis with human-readable explanations, Octane bridges the gap between high-signal tooling and real-world developer workflows. Our platform provides contextualized vulnerability reports, clustered findings, and AI-enhanced guidance to make security reviews faster, clearer, and more actionable.

Octane was designed in collaboration with top auditors and engineers to meet the unique demands of web3 and next-gen software systems. We work closely with security researchers, protocol teams, and developer-first organizations to continuously improve our threat detection models and surface what actually matters.

Our clients include leading teams across DeFi, L2 protocols, infrastructure tooling, and security research. Octane integrates seamlessly with developer workflows—from GitHub automation to custom feedback tooling—so teams can reduce noise, eliminate risk, and ship with confidence.

Octane is backed by deep domain knowledge in static analysis, program synthesis, and exploit modeling, with capabilities designed for both technical depth and usability.

To learn more or request a demo, visit www.octane.security, or reach out to us at info@octane.security.

Octane Security
Distributed team with a global presence
www.octane.security

"Summary of findings"

This review analyzed the Superform: v2-periphery (main) smart contracts using Octane’s automated analysis and included team feedback on findings.

The scan identified a total of 0 issues.

Vulnerabilities Confirmed	00/00
Critical	00/00
High	00/00
Medium	00/00
Low	00/00
Informational	00/00

Vulnerabilities & Warnings

Warning #1

Zeroed hook-update timelock due to missing proxy initialization in SuperVaultAggregator causes immediate hook authorization updates enabling rapid fund exfiltration

Severity: MEDIUM

Likelihood: LOW

Impact: HIGH

Status: Acknowledged

Assumes a malicious strategist

When SuperVaultAggregator is deployed behind a proxy without initializing storage, `_hooksRootUpdateTimelock` remains 0 in proxy storage. This removes the intended 15-minute delay for global/strategy hook root updates, allowing authorized roles to propose and immediately execute root changes and then immediately execute hooks. In case of manager/governance compromise or malice, this enables rapid fund movement from strategies without a monitoring window.

SuperVaultAggregator defines a storage variable `_hooksRootUpdateTimelock` intended to default to 15 minutes and used in `proposeGlobalHooksRoot` and `proposeStrategyHooksRoot` to set an `effectiveTime = block.timestamp + _hooksRootUpdateTimelock`. When used behind a proxy with no initializer that sets storage in the proxy context, this variable stays 0 in proxy storage. As a result, `effectiveTime` equals `block.timestamp`, and `executeGlobalHooksRootUpdate` / `executeStrategyHooksRootUpdate` can succeed immediately. SuperVaultStrategy.executeHooks relies on SuperVaultAggregator.validateHook (Merkle proof against global or strategy roots) and then executes arbitrary external calls built by the registered hook (only calls back to the aggregator are blocked). With a zero timelock, an authorized-but-malicious or compromised role (mainManager for strategy roots; SUPER_GOVERNOR for global root) can same-block: (1) propose a root that authorizes a malicious hook+args, (2) execute the root update, and (3) execute hooks to move strategy-held assets, removing any detection/response window. Victims are users who deposited into affected strategies. Mitigation requires governance to call `setHooksRootUpdateTimelock` to a non-zero value post-deploy; otherwise, the system operates without the intended delay.

file: src/SuperVault/SuperVaultAggregator.sol

```
78     uint256 private constant _MANAGER_CHANGE_TIMELOCK = 7 days;
79     uint256 private _hooksRootUpdateTimelock = 15 minutes;

80
```

```

81  // Timelock for parameter changes (3 days)

...

891  }
892

893  // Update the timelock
894  _hooksRootUpdateTime = newTimelock;

...

906  // Set new root with timelock
907  _proposedGlobalHooksRoot = newRoot;
908 - uint256 effectiveTime = block.timestamp + _hooksRootUp-
    dateTimelock;

909  _globalHooksRootEffectiveTime = effectiveTime;

...

964  // Set proposed root with timelock
965  _strategyData[strategy].proposedHooksRoot = newRoot;
966 - uint256 effectiveTime = block.timestamp + _hooksRootUp-
    dateTimelock;

967  _strategyData[strategy].hooksRootEffectiveTime = effec-
    tiveTime;

...

1146 /// @inheritdoc ISuperVaultAggregator
1147 function getHooksRootUpdateTime() external view returns
    (uint256) {
1148- return _hooksRootUpdateTime;

1149 }

```

```

78     uint256 private constant _MANAGER_CHANGE_TIMELOCK = 7 days;
79     uint256 private _hooksRootUpdateTimelock = 15 minutes;
80 + bool private _hooksRootUpdateTimelockInitialized;
81
82     // Timelock for parameter changes (3 days)
83
84     ...
85
892 }
893
894 +_hooksRootUpdateTimelockInitialized = true;
895 // Update the timelock
896 _hooksRootUpdateTimelock = newTimelock;
897
898 ...
899
908 // Set new root with timelock
909 _proposedGlobalHooksRoot = newRoot;
910 + uint256 effectiveTime = block.timestamp +
911 +(
912 +_hooksRootUpdateTimelockInitialized
913 +? _hooksRootUpdateTimelock
914 +: 15 minutes
915 +);
916 _globalHooksRootEffectiveTime = effectiveTime;
917
918 ...
919
971 // Set proposed root with timelock
972 _strategyData[strategy].proposedHooksRoot = newRoot;
973 + uint256 effectiveTime = block.timestamp +
974 +(
975 +_hooksRootUpdateTimelockInitialized
976 +? _hooksRootUpdateTimelock
977 +: 15 minutes
978 +);
979 _strategyData[strategy].hooksRootEffectiveTime = effectiveTime;
980
981 ...
982
1158 /// @inheritdoc ISuperVaultAggregator
1159 function getHooksRootUpdateTimelock() external view returns
    (uint256) {

```

```
1160+ return
1161+ _hooksRootUpdateTimelockInitialized
1162+? _hooksRootUpdateTimelock
1163+ : 15 minutes;
1164 }
```

Octane Security

Severity

Impact Explanation: [High] Authorized root changes with zero timelock enable immediate hook executions that can directly and materially transfer strategy-held principal funds to attacker-controlled destinations.

Likelihood Explanation: [Low] Exploitation requires both a proxied deployment without initializing the timelock (mis-configuration) and misuse/compromise of a trusted role (mainManager or governance), making it a rare combination of preconditions.

Exploitation

Exploitation Scenarios:

Malicious mainManager: Aggregator behind proxy with `_hooksRootUpdateTimelock=0`; attacker controlling a strategy's mainManager proposes a strategy hooks root that authorizes a dangerous hook+args, immediately executes the root update, then calls `executeHooks` to move strategy funds to attacker-controlled destinations.

Compromised SUPER_GOVERNOR: Aggregator behind proxy with `_hooksRootUpdateTimelock=0`; attacker with SUPER_GOVERNOR proposes and immediately executes a global hooks root that authorizes dangerous hook+args; managers across multiple strategies then execute hooks to move assets quickly.

Same-block propose-and-execute: Aggregator behind proxy with `_hooksRootUpdateTimelock=0`; an authorized attacker proposes and executes a root update and then immediately calls `executeHooks` in the same block/MEV bundle, leaving guardians/monitors no window to react before funds are moved.

Warning #2

Non-binding hook validation and unbounded ETH/token forwarding in Bank._executeHooks enables draining SuperBank funds

Severity: MEDIUM

Likelihood: LOW

Impact: HIGH

Status: Acknowledged

assumes a malicious trusted role who injects a malicious hook

Bank executes arbitrary Execution steps returned by registered hooks, forwarding specified ETH values and callData without caps or correlation to msg.value. The Merkle proof validates only inspect(data) rather than the actual execution steps, and the slippage check relies on hook-reported getOutAmount. A malicious or compromised registered hook can pass validation and transfer SuperBank's ETH and ERC20 balances to an attacker.

In Bank._executeHooks, the contract iterates over Execution[] returned by a hook and performs low-level calls that forward the exact (target, value, callData) provided, spending from the Bank's balance with no per-call or total cap and no linkage to msg.value. Hook validation checks only a Merkle proof over (hookAddress, inspect(hookData)), not the actual Execution steps, and slippage protection uses hook-controlled getOutAmount. As a result, a registered hook that is malicious or becomes compromised (e.g., via upgrade) can return execution steps that transfer all ETH and ERC20 tokens held by SuperBank to an attacker, while still passing Merkle and slippage checks. onlyBankManager gating limits who can trigger execution but does not mitigate a malicious hook once executed.

file: src/Bank.sol

```
110     executions = hook.build(prevHook, address(this),
111         hookData);

112     uint256 len = executions.length;

    ...
```

```
127 // Execute the call
```

```
128 // We call via assembly to avoid memcpying the  
returndata
```

```
129 assembly {
```

```
110 executions = hook.build(prevHook, address(this),  
hookData);
```

```
111
```

```
112 +// SECURITY TODOs:
```

```
113 +// - Enforce pre/post shape: executions[0] must be  
preExecute(...) with value=0, and last must be postExecute(...) with value=0.
```

```
114 +// - Validate all execution targets/selectors/values  
against a governance-registered validator (do not rely on hook.in-  
spect()).
```

```
115 +// - Bind allowed targets via Merkle-approved config  
from an external validator; reject unknown targets.
```

```
116 +// - Cap native ETH outflow (e.g., require  
sum(execution.value) <= msg.value or a declared per-hook budget)  
to avoid draining existing ETH.
```

```
117 +// - Consider pinning hook extcodehash at registration  
to prevent upgrades of approved hooks without governance.
```

```
118 uint256 len = executions.length;
```

```
...
```

```
133 // Execute the call
```

```
134 +// SECURITY: Before executing, ensure this step  
passes validator checks (target, selector, value).
```

```
135 +// SECURITY: If using ETH caps, accumulate total  
value and enforce budget limits before/after the loop.
```

```
136 +// SECURITY: Reject any step that attempts to send  
ETH to non-approved recipients.
```

```
137 // We call via assembly to avoid memcpying the  
returndata
```

```
138 assembly {
```

Severity

Impact Explanation: [High] Direct, material loss of principal funds (SuperBank's ETH and ERC20 assets) is possible, disrupting core protocol fund distribution.

Likelihood Explanation: [Low] Exploitation relies on trusted components or roles: a registered hook must be malicious or become compromised (e.g., via upgrade/admin key), and a BankManager must execute it as part of routine operations.

Exploitation

Exploitation Scenarios:

Compromised upgradeable registered hook drains all ERC20 and ETH: Preconditions: a registered hook with a Merkle root exists; the hook is upgradable or becomes compromised; SuperBank holds ERC20 (e.g., UP) and ETH; a BankManager executes executeHooks with valid data/proofs. The malicious hook returns inspect(data) matching the whitelisted leaf, and build() returns Execution[] that transfers all SuperBank tokens and ETH to the attacker; getOutAmount is faked to satisfy slippage. Bank executes the calls and funds are drained.

ETH-only drain via malicious hook: Preconditions: a registered hook exists; SuperBank has non-zero ETH; a BankManager executes executeHooks. The hook passes Merkle via inspect(data) and returns an Execution step sending address(SuperBank).balance ETH to the attacker, followed by benign steps; getOutAmount is faked. Bank forwards the ETH and the slippage check passes.

ERC20-only drain via malicious hook: Preconditions: a registered hook exists; SuperBank holds UP or other ERC20 balances; a BankManager executes executeHooks. The hook passes Merkle and returns token.transfer calls moving the entire ERC20 balances from SuperBank to the attacker; getOutAmount is faked to pass slippage. Bank executes and funds are drained.



Octane Security



Warning #3

Lack of safeguards against weird-ERC20 behavior in SuperVault escrow redemption causes frozen funds/DoS

Severity: MEDIUM

Likelihood: LOW

Impact: HIGH

Status: Acknowledged

requires weird ERC20 which we won't support

Redemption depends on `escrow.returnAssets` performing an ERC20 transfer. If the underlying token blocks or under-delivers transfers (e.g., blacklist/freeze/upgrade or fee-on-transfer), users can be denied redemptions after their shares were already burned, with no rescue path.

`SuperVaultStrategy.fulfillRedeemRequests` burns users' shares in the escrow and transfers assets from the strategy to the escrow. Users then claim via `SuperVault.withdraw/redeem`, which call `ISuperVaultEscrow.returnAssets(receiver, assets)`. The escrow uses `SafeERC20.safeTransfer` of the underlying token. If the token later prevents transfers from the escrow (blacklist/freeze/upgrade) the transfer reverts and the claim is permanently blocked because shares are already burned and there is no rescue function. Separately, for fee-on-transfer tokens, the strategy does not verify the amount actually received by the escrow during fulfillment; per-user claimable amounts can exceed the escrow's real balance, causing late claimants to hit `NOT_ENOUGH_ASSETS` and face DoS until the manager tops up. The aggregator does not restrict asset types, and escrow has no emergency withdrawal, making this a design-level liveness risk under weird-ERC20 behaviors.

file: `src/SuperVault/SuperVaultStrategy.sol`

```
417 }
418
419 -// Burn shares
420 - ISuperVault(_vault).burnShares(vars.totalRequestedShares);
421 -
422 -// Transfer net assets to escrow
423   if (vars.totalNetAssetsOut > 0) {
424   -_asset.safeTransfer(
425   - ISuperVault(_vault).escrow(),
426   - vars.totalNetAssetsOut
```

```
427 );
```

```
428 }
```

```
429
```

```
430 emit RedeemRequestsFulfilled(
```

```
417 }
```

```
418
```

```
419 +// Transfer net assets to escrow first and verify full  
    receipt, then burn shares
```

```
420 if (vars.totalNetAssetsOut > 0) {
```

```
421 + address escrowAddr = ISuperVault(_vault).escrow();
```

```
422 + uint256 escrowBefore = _getTokenBalance(
```

```
423 + address(_asset),
```

```
424 + escrowAddr
```

```
425 );
```

```
426 + _asset.safeTransfer(escrowAddr, vars.totalNetAssetsOut);
```

```
427 + uint256 received = _getTokenBalance(address(_asset),  
    escrowAddr) -
```

```
428 + escrowBefore;
```

```
429 + if (received != vars.totalNetAssetsOut) revert  
    OPERATION_FAILED();
```

```
430 }
```

```
431 +// Burn shares after successful transfer
```

```
432 + ISuperVault(_vault).burnShares(vars.totalRequestedShares);
```

```
433
```

```
434 emit RedeemRequestsFulfilled(
```

Severity

Impact Explanation: [High] Funds can be frozen/blocked for longer than a week with no workaround and core redemption functionality can become unusable.

Likelihood Explanation: [Low] Scenarios require trusted token admin actions (freeze/upgrade) or uncommon token mechanics (fee-on-transfer) and, for persistence in the fee-on-transfer case, operator inaction; these are multiplicative preconditions and not generally present.

Exploitation

Exploitation Scenarios:

Centrally-controlled token freezes escrow: After `fulfillRedeemRequests` burns shares and sends assets to the escrow, the token admin blacklists or freezes the escrow. Subsequent `withdraw/redeem` calls revert in `escrow.returnAssets` due to `safeTransfer` failing, permanently blocking users' claims with no rescue path.

Fee-on-transfer token underfunds escrow: During `fulfillRedeemRequests` the strategy transfers the sum of users' claimable assets to the escrow, but a transfer fee reduces the received amount; the strategy does not verify escrow receipt. Early claimants succeed; late claimants face `NOT_ENOUGH_ASSETS` and are DoS'd until the manager tops up. Shares were already burned at fulfillment.

Upgradeable token changes behavior post-fulfillment: After assets sit in escrow post-fulfillment, the token is upgraded or reconfigured to block or fully tax transfers from the escrow. `escrow.returnAssets` then reverts, permanently blocking user redemptions with no fallback.

Warning #4

Documentation/NatSpec mismatch in ISuperOracle interface causes off-chain mis-decoding and missed monitoring enabling fee overpayment and temporary pauses

Severity: LOW

Likelihood: LOW

Impact: MEDIUM

Status: Fixed

Fixed

ISuperOracle's NatSpec comments for key events and a function omit/mis-order the quotes parameter compared to actual signatures. While on-chain behavior and ABI decoding are correct, off-chain systems that rely on the comments (instead of the ABI) can miss or misinterpret oracle configuration events, reducing monitoring effectiveness and, when combined with privileged governance misuse, allowing unnoticed oracle changes that can inflate upkeep costs and temporarily pause strategies.

In ISuperOracle.sol, NatSpec comments for OraclesConfigured, OracleUpdateQueued, and OracleUpdateExecuted omit the quotes parameter and imply a different order than the actual event declarations. The queueOracleUpdate function's NatSpec also mis-orders providers and quotes. The implementation in SuperOracleBase uses and emits the correct signatures. Standard tooling decodes from the ABI, so on-chain correctness is unaffected. However, off-chain consumers that ignore ABIs and follow the comments may miss events (topic mismatch) or mislabel fields. If a privileged role (via SuperGovernor) pushes harmful oracle updates unnoticed during the timelock, SuperGovernor._convertGasToUp can compute inflated upkeep costs. SuperVaultAggregator.forwardPPS may then over-deduct upkeep, draining strategy upkeep balances and auto-pausing strategies when balances become insufficient. Impacts are limited to fees and temporary availability loss; no principal loss or broken core on-chain logic.

file: src/interfaces/oracles/ISuperOracle.sol

```
68  /// @notice Emitted when oracles are configured
69  /// @param bases Array of base assets

70  /// @param providers Array of provider indexes
71  /// @param feeds Array of oracle addresses

...
```

```

80  /// @param bases Array of base assets
81  /// @param providers Array of provider indexes

82  /// @param feeds Array of oracle addresses
83  /// @param timestamp Timestamp when update was queued

...

93  /// @param bases Array of base assets
94  /// @param providers Array of provider indexes

95  /// @param feeds Array of oracle addresses
96  event OracleUpdateExecuted(

...

186  /// @notice Queue oracle update for timelock
187  /// @param bases Array of base assets

188  /// @param providers Array of provider ids
189 - /// @param quotes Array of quote assets
190  /// @param feeds Array of oracle addresses
191  function queueOracleUpdate(

```

```

68  /// @notice Emitted when oracles are configured
69  /// @param bases Array of base assets
70 + /// @param quotes Array of quote assets
71  /// @param providers Array of provider indexes
72  /// @param feeds Array of oracle addresses

...

81  /// @param bases Array of base assets
82  /// @param providers Array of provider indexes
83 + /// @param quotes Array of quote assets
84  /// @param feeds Array of oracle addresses
85  /// @param timestamp Timestamp when update was queued

...

95  /// @param bases Array of base assets
96  /// @param providers Array of provider indexes
97 + /// @param quotes Array of quote assets
98  /// @param feeds Array of oracle addresses

```

```
99     event OracleUpdateExecuted(  
  
    ...  
  
189    /// @notice Queue oracle update for timelock  
190    /// @param bases Array of base assets  
191 +/// @param quotes Array of quote assets  
192    /// @param providers Array of provider ids  
  
193    /// @param feeds Array of oracle addresses  
194    function queueOracleUpdate(  

```

Severity

Impact Explanation: [Medium] Potential direct, material loss of fees (inflated upkeep deductions) and significant but temporary availability loss (auto-pauses), with no principal loss or broken core logic.

Likelihood Explanation: [Low] Requires both a privileged governance misuse/compromise to push harmful oracle changes and an operator mistake where off-chain systems ignore the ABI and rely on comments; timelocks also provide a review window.

Exploitation

Exploitation Scenarios:

Monitoring derived from comments misses OracleUpdate-Queued/Executed due to wrong signature (quotes omitted). A privileged actor queues and executes harmful oracle updates during the 1-week timelock without alerts. Result: inflated upkeep costs cause excess fee deductions and eventual auto-pause of strategies when upkeep is insufficient.

An operator UI decodes fields based on comments, mislabeling parameters and omitting quotes. Operators false-assure on pending oracle changes and do not intervene. After execution, upkeep conversions become inflated, leading to higher fee deductions and temporary strategy pauses.

A policy watcher ignores the quotes array (per comments) and fails to detect new base/quote/provider pairs. A privileged actor introduces unwanted pairs and executes the change. Subsequent upkeep conversions reflect manipulated pricing, increasing upkeep spending and triggering temporary pauses when balances run low.

Warning #5

Unbounded enumeration in SuperVaultAggregator registry getters causes off-chain enumeration DoS and can indirectly block core operations

Severity: LOW

Likelihood: LOW

Impact: MEDIUM

Status: Fixed

Fixed

The `getAllSuperVaults/getAllSuperVaultStrategies/getAllSuperVaultEscrows` getters return entire `EnumerableSets` via `values()`, an unbounded $O(n)$ copy that can revert for large sets. If off-chain services or on-chain integrators misuse these getters for enumeration, an attacker can bloat the registry (via `createVault`) to cause enumeration failures, potentially stalling PPS updates and temporarily blocking core operations due to `PPS_EXPIRED`.

`SuperVaultAggregator` exposes `getAllSuperVaults/getAllSuperVaultStrategies/getAllSuperVaultEscrows` that return full sets using `EnumerableSet.values()`, an $O(n)$ storage-to-memory copy with unbounded cost. `createVault` is permissionless and adds to these sets without a cap, enabling an attacker to inflate them. While the Aggregator does not enumerate these sets in state-changing paths, off-chain services or integrators that use these getters for enumeration can face `eth_call` reverts/timeouts when the sets grow large. If the PPS updater depends on these getters, PPS updates can stall; `SuperVaultStrategy` then reverts operations after `ppsExpiration` (default 1 day) with `PPS_EXPIRED`, temporarily blocking deposits, mints, redeem fulfillment, and fee skimming. Safe alternatives exist (event indexing, index-based getters, pagination), and on-chain protocol logic remains unaffected directly. Thus, the issue is a Low-severity, integration-driven DoS risk rather than a direct on-chain vulnerability.

file: `src/interfaces/SuperVault/ISuperVaultAggregator.sol`

```
917     function superVaultEscrows(uint256 index) external view returns
      (address);
918
919     /// @notice Validates a hook against both global and
      strategy-specific Merkle roots
920     /// @param strategy Address of the strategy
```

```
917     function superVaultEscrows(uint256 index) external view returns
      (address);
918
919 +   function getSuperVaultsCount() external view returns (uint256);
920 +   function getSuperVaultStrategiesCount() external view returns
      (uint256);
921 +   function getSuperVaultEscrowsCount() external view returns
      (uint256);
922     /// @notice Validates a hook against both global and
      strategy-specific Merkle roots
923     /// @param strategy Address of the strategy
```

Severity

Impact Explanation: [Medium] Significant but temporary availability loss of core protocol operations (deposits, mints, redeem fulfillment, fee skimming) when PPS updates stall and Strategies revert with PPS_EXPIRED; no direct principal loss and clear operational workaround exists.

Likelihood Explanation: [Low] Exploitation depends on off-chain operator/integrator misuse (enumerating via unbounded getters) and attacker-funded registry bloat; straightforward mitigations (event indexing, index-based getters, pagination, increased eth_call limits) exist, making this unlikely under competent operational practices.

Exploitation

Exploitation Scenarios:

An off-chain PPS updater relies on `getAllSuperVaultStrategies()` to enumerate strategies. An attacker repeatedly calls `createVault` to bloat the registry. The updater's `eth_call` to `getAllSuperVaultStrategies()` reverts due to large array size, PPS updates stall, and after ~1 day SuperVaultStrategy starts reverting core operations with PPS_EXPIRED until the operator switches to event/indexed pagination or raises call limits.

A third-party on-chain integrator uses `getAllSuperVaultStrategies()` inside a state-changing function to batch actions. An attacker bloats the registry via `createVault`, causing the integrator's transaction to exceed block gas/memory and revert, making the integrator's function uncallable.

A UI/monitoring tool fetches lists via `getAllSuperVaults/getAllSuperVaultStrategies`. An attacker bloats the registry, the `eth_call` reverts or times out, and dashboards fail or become incomplete, degrading UX and observability.

Warning #6

Insufficient-gas catch and returns-block reverts in SuperOracle average provider path cause DoS of average price queries

Severity: INFORMATIONAL

Likelihood: LOW

Impact: INFORMATIONAL

Status: Fixed

Fixed

Average provider price queries can revert despite `revertOnError=false` because the catch blocks unconditionally revert on low gas (EIP-150 threshold) and uncatchable reverts can occur in the returns block (token decimals decoding/overflow), enabling a single misbehaving feed or misconfigured token to DoS view-only price queries.

In `SuperOracleBase` and `SuperOracleL2`, `_getQuoteFromOracle` wraps external calls to `latestRoundData/decimals` in try/catch, but the catch path reverts with `INSUFFICIENT_GAS_FOR_EXTERNAL_CALL` when `gasleft()` 255, or `uint8` overflow in the addition) occurs in the returns block and is not caught, aborting the average query. In this repository, oracle quotes are only used in view paths (e.g., `SuperGovernor.getUpkeepCostPerSingleUpdate` via `_convertGasToUp`), so the impact is view-only DoS. Governance mitigations exist (1-hour timelock for provider removals, 1-week timelock for additions), but DoS persists until removal/reconfiguration is executed.

file: `src/oracles/SuperOracleBase.sol`

```
453 // EIP-150: Ensure at least 1/64 of gas remained to
    // prevent out-of-gas reverts being misinterpreted as
454 // oracle failures
455 - if (gasleft() <= gasBefore / 64)
456     revert INSUFFICIENT_GAS_FOR_EXTERNAL_CALL();
...

490 // EIP-150: Ensure at least 1/64 of gas remained to
    // prevent out-of-gas reverts being misinterpreted as
491 // oracle failures
492 - if (gasleft() <= gasBefore / 64)
493     revert INSUFFICIENT_GAS_FOR_EXTERNAL_CALL();
```



```
453 // EIP-150: Ensure at least 1/64 of gas remained to
    prevent out-of-gas reverts being misinterpreted as
454 // oracle failures
455 + if (gasleft() <= gasBefore / 64 && revertOnError)
456     revert INSUFFICIENT_GAS_FOR_EXTERNAL_CALL();

...

490 // EIP-150: Ensure at least 1/64 of gas remained to
    prevent out-of-gas reverts being misinterpreted as
491 // oracle failures
492 + if (gasleft() <= gasBefore / 64 && revertOnError)
493     revert INSUFFICIENT_GAS_FOR_EXTERNAL_CALL();
```

Severity

Impact Explanation: [Informational] In this repository, the affected quoting paths are used only by view functions and do not impact on-chain state or funds; thus the impact is a view-only availability disruption.

Likelihood Explanation: [Low] Scenarios require a misbehaving external integration (oracle or uptime feed) or trusted role misconfiguration of tokens; these are treated as low-likelihood conditions and primarily enable griefing rather than profit.

Exploitation

Exploitation Scenarios:

A configured data oracle gas-bombs and reverts on `latestRoundData` or `decimals`; the catch sees `gasleft() <= gasBefore/64` and reverts with `INSUFFICIENT_GAS_FOR_EXTERNAL_CALL`, causing `getQuoteFromProvider(..., AVERAGE_PROVIDER)` to revert instead of skipping the provider, resulting in view-only DoS for that pair.

On L2, a shared uptime oracle gas-bombs and reverts on `latestRoundData`; `SuperOracleL2`'s catch reverts with `INSUFFICIENT_GAS_FOR_EXTERNAL_CALL` before reaching data feeds, producing a broader view-only DoS across pairs that rely on that uptime oracle.

A misconfigured/malicious ERC20 used as base or quote returns a `decimals()` value that causes `abi.decode(uint8)` to revert or triggers `uint8` overflow in `feedDecimals + baseDecimals`; these reverts occur after a successful aggregator call and are not caught, causing average quoting to revert for that pair.

Warning #7

Missing reconciliation for escrow funding with non-standard ERC20s in SuperVaultStrategy/SuperVault causes redeem/withdraw DoS

Severity: INFORMATIONAL

Likelihood: MEDIUM

Impact: HIGH

Status: Acknowledged

Assume non standard ERC20 which we won't support so out of scope

When the underlying asset has non-standard ERC20 behavior (fee-on-transfer, rebasing, or admin-controlled balance changes), the escrow can receive fewer tokens than credited to users' entitlements during fulfill, causing legitimate redeem/withdraw calls to revert due to an escrow balance shortfall.

The redemption flow credits per-user entitlements (`state.maxWithdraw`) during `SuperVaultStrategy.fulfillRedeemRequests` by increasing `maxWithdraw` and then transferring the aggregated amount to the escrow. The code assumes standard ERC20 behavior and does not reconcile how many tokens the escrow actually receives. If the underlying token is fee-on-transfer, rebasing, or subject to admin balance controls, the escrow may end up with fewer tokens than the sum of users' entitlements.

Subsequently, `SuperVault.withdraw/redeem` enforces two checks: (1) `assets <= maxWithdraw(controller)`, and (2) `assets <= escrow's token balance`. Under short-fall conditions, a user can pass the first check but fail the second, causing a revert (`NOT_ENOUGH_ASSETS`). This results in a functional denial-of-service for withdrawals/redemptions in that vault until the escrow is topped up or the discrepancy is otherwise resolved.

The system does not restrict assets to standard ERC20s at vault creation, so these conditions are reachable in deployments that choose such tokens. The issue does not cause value leakage but can block users from claiming their owed assets.

file: `src/SuperVault/SuperVaultStrategy.sol`

```
422 // Transfer net assets to escrow
423 if (vars.totalNetAssetsOut > 0) {
424   _asset.safeTransfer(
425     ISuperVault(_vault).escrow(),
426     vars.totalNetAssetsOut
427   );
```

```
428 }
```

```
422 // Transfer net assets to escrow
423 if (vars.totalNetAssetsOut > 0) {
424 +// Reconcile escrow received amount to guard against
    non-standard ERC20 behavior (e.g., fee-on-transfer)
425 + address escrowAddr = ISuperVault(_vault).escrow();
426 + uint256 beforeBal = _getTokenBalance(address(_asset),
    escrowAddr);
427 + _asset.safeTransfer(escrowAddr, vars.totalNetAssetsOut);
428 + uint256 afterBal = _getTokenBalance(address(_asset),
    escrowAddr);
429 + if (afterBal < beforeBal) revert INSUFFICIENT_LIQUID-
    ITY();
430 + if (afterBal - beforeBal != vars.totalNetAssetsOut) {
431 + revert INSUFFICIENT_LIQUIDITY();
432 +}
433 }
```



Severity

Impact Explanation: [High] For affected vaults, core withdrawal/redemption functionality can be rendered unusable and user funds effectively frozen until the escrow is replenished or the discrepancy is resolved.

Likelihood Explanation: [Medium] It requires selection of a non-standard ERC20 as the underlying asset and user participation in such a vault, which is plausible in permissionless deployments but not universal.

Exploitation

Exploitation Scenarios:

Fee-on-transfer token: A vault uses a fee-on-transfer asset. During fulfill, the strategy credits user entitlements (`maxWithdraw`) and transfers `totalAssetsOut` to escrow, but the escrow receives less due to transfer fees. Later, a user's redeem passes the per-user entitlement check but fails the escrow balance check and reverts, blocking claims.

Negative-rebasing token: After fulfill funds the escrow and entitlements are set, a negative rebase reduces escrow's balance. Users attempting to redeem amounts equal to their entitlements pass the `maxWithdraw` check but fail the escrow balance check, causing reverts.

Admin-controlled token: A token with admin controls reduces or freezes the escrow's balance after fulfill. Users' entitlements remain unchanged, but the escrow holds fewer tokens, causing legitimate withdraw/redeem calls to revert.